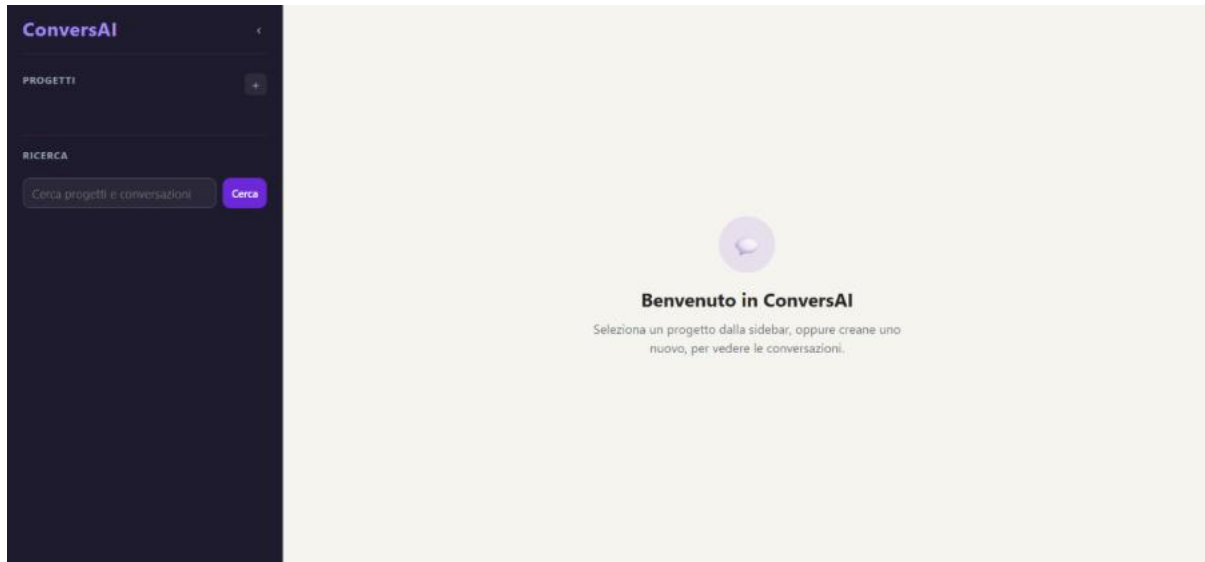


ConversAI

Relazione del progetto di Applicazioni Web
Giulia Anedda Matricola 169298 A. A. 2025/26



1. Architettura dell'applicazione

ConversAI è un'applicazione web full-stack su tre livelli. Il frontend è una Single Page Application scritta in HTML, CSS e JavaScript fornita come file statico da Express. Il backend gira su Node.js con Express e si occupa delle API REST e delle risposte simulate. Per i dati ho usato SQLite tramite il pacchetto sqlite3.

Il browser e il server si parlano solo via HTTP: il frontend usa `fetch()` per le richieste, il server risponde sempre in JSON. Non è necessario ricaricare la pagina, ogni operazione aggiorna il DOM direttamente.

Per il layout ho scelto di tenere la sidebar dedicata solo ai progetti, mentre la lista delle conversazioni e la chat si trovano nell'area principale. All'inizio avevo provato a mettere anche le conversazioni nella sidebar, ma, a mio avviso, non risultava intuitivo nell'utilizzo. Spostandole nell'area principale i filtri si leggono meglio e la separazione tra navigazione a sinistra e contenuto a destra risulta molto più pulita.

2. Struttura del database

Il database contiene tre tabelle con relazioni uno-a-molti; infatti, un progetto ha molte conversazioni e una conversazione ha molti messaggi. Le tabelle vengono create automaticamente all'avvio tramite `CREATE TABLE IF NOT EXISTS` in `database.js`.

Tabella	Colonne principali
projects	id (PK), name NOT NULL, description, created_at, updated_at
conversations	id (PK), project_id (FK), topic NOT NULL, is_favorite (0/1), is_archived (0/1), created_at, updated_at
messages	id (PK), conversation_id (FK), role ('user'/'bot'), content NOT NULL, created_at

dove PK = primary key e FK = foreign key

Istruzioni SQL per ricreare il database

```
CREATE TABLE IF NOT EXISTS projects (
  id          INTEGER PRIMARY KEY,
  name        TEXT NOT NULL,
  description  TEXT,
  created_at  TEXT NOT NULL,
  updated_at  TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS conversations (
  id          INTEGER PRIMARY KEY,
  project_id  INTEGER NOT NULL,
  topic       TEXT NOT NULL,
  is_favorite INTEGER NOT NULL DEFAULT 0,
  is_archived INTEGER NOT NULL DEFAULT 0,
  created_at  TEXT NOT NULL,
  updated_at  TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS messages (
  id          INTEGER PRIMARY KEY,
  conversation_id INTEGER NOT NULL,
  role        TEXT NOT NULL,
  content     TEXT NOT NULL,
  created_at  TEXT NOT NULL
);
```

3. Endpoint REST

Tutte le risposte sono in formato JSON. I codici di errore usati sono:

- 400 per dati non validi o mancanti;
- 404 per risorse inesistenti;
- 500 per errori interni del database.

Progetti

Metodo	Path	Dati ricevuti	Risposta	Errori
GET	/api/projects	—	200: array di tutti i progetti	—
POST	/api/projects	Nome (obbligatorio), descrizione (opzionale)	201: progetto creato + header Location	400 se manca nome
GET	/api/projects/:id	—	200: oggetto progetto	404 se non trovato
PATCH	/api/projects/:id	Nome (obbligatorio), descrizione (opzionale)	200: progetto aggiornato	400 e 404
DELETE	/api/projects/:id	—	204: elimina a cascata conversazioni e messaggi	404 se non trovato

Conversazioni

Metodo	Path	Dati ricevuti	Risposta	Errori
GET	/api/projects/:id/conversations	?favorite=true / ?archivedOnly=true	200: array conversazioni filtrate	404 progetto
POST	/api/projects/:id/conversations	topic (obbl.)	201: conversazione creata + header Location	400 / 404
GET	/api/conversations/:id	—	200: oggetto conversazione	404
PATCH	/api/conversations/:id	topic, isFavorite, isArchived (opz.)	200: conversazione aggiornata	400 / 404
DELETE	/api/conversations/:id	—	204: elimina anche i messaggi	404

Messaggi e Ricerca

Metodo	Path	Dati ricevuti	Risposta	Errori
GET	/api/conversations/:id/messages	—	200: array messaggi in ordine cronologico	404
POST	/api/conversations/:id/messages	content (obbl.)	201: { msgUtente, msgBot } (vedi esempio sotto)	400 / 404
GET	/api/search?q=...	q: stringa di ricerca	200: { progetti, conversazioni, messaggi }	400 se q mancante

Esempio di risposta POST /api/conversations/:id/messages:

```
{
  "msgUtente": { "id": 5, "role": "user", "content": "Ciao!", "createdAt": "..."},
  "msgBot":     { "id": 6, "role": "bot",  "content": "Ciao! Come posso aiutarti?",
  "createdAt": "..."}
}
```

4. Flusso di comunicazione

Il frontend usa `fetch()` con `.then()` e `.catch()` per tutte le chiamate HTTP. La funzione `leggiJson(r)` serve a leggere la risposta del server in un solo posto: restituisce `null` per le risposte 204, altrimenti chiama `.json()` e lancia un errore se la risposta non è corretta.

Flusso tipico per l'invio di un messaggio:

- L'utente scrive nel form e preme Invio, `inviaMessaggio()` disabilita il pulsante Invia e chiama `POST /api/conversations/:id/messages`.
- Il server riceve il testo, lo salva nel database come messaggio utente, chiama `rispostaBot()` per generare la risposta, salva anche questa e restituisce `{ msgUtente, msgBot }`.
- La funzione `creaChat()` costruisce le due chat nel DOM e il pulsante viene riabilitato.

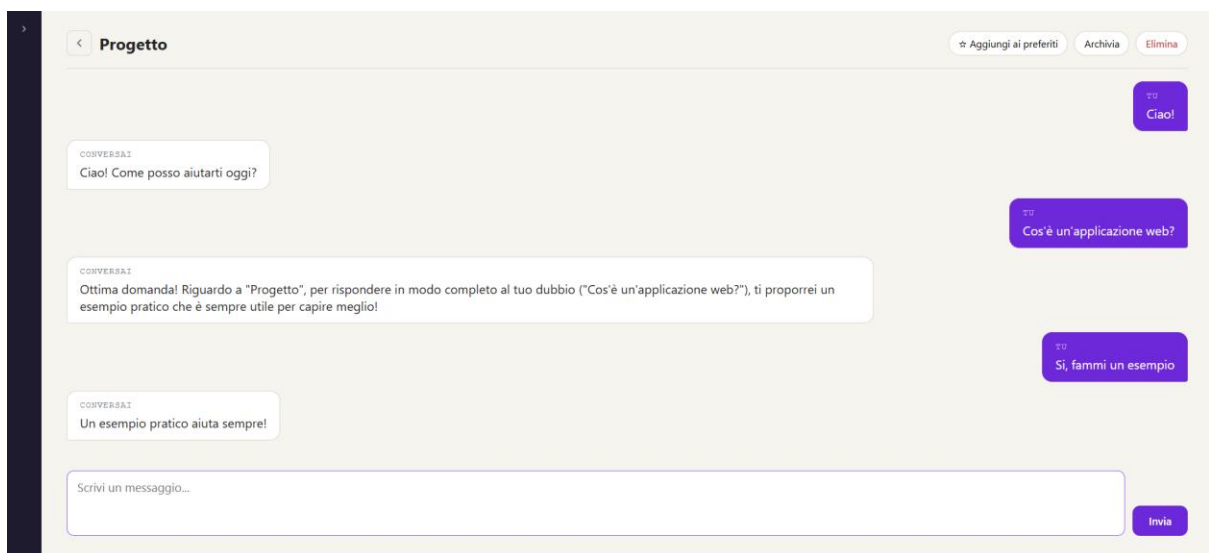
La codifica dell'URL di ricerca usa `codificaQuery` (testo), che scorre il testo carattere per carattere e sostituisce i caratteri speciali (`%`, spazio, `&`, `#`, `?`, `=`, `+`) con i rispettivi codici percent-encoded.

5. Simulazione della risposta

La funzione `rispostaBot(testo, topic)` in `server.js` analizza il messaggio tramite espressioni regolari. Le regole vengono controllate in ordine; viene applicata la prima che corrisponde:

Condizione	Pattern
Saluto	ciao, Ciao, buongiorno, Buongiorno, buonasera, Buonasera, salve, Salve
Ringraziamento	grazie, Grazie
Domanda	presenza del carattere “?”
Sintesi	riassunto, Riassunto, sintesi, Sintesi, summary, Summary, Riassumi, riassumi
Spiegazione	spiega, Spiega, explain, Explain, spiegami, Spiegami
Esempio	esempio, Esempio, example, Example
Default	(qualsiasi altro testo)

Se nessuna regola corrisponde, la funzione restituisce comunque una risposta generica che riporta il testo ricevuto e chiede di specificare meglio la richiesta.



6. Funzionalità aggiuntive

Ricerca globale

La funzione `cerca()` in `database.js` carica tutte le righe delle tre tabelle con tre `SELECT` separate e le filtra in JavaScript tramite `cercaStringa(testo, query)`, un ciclo `for` annidato che confronta i caratteri uno a uno. La ricerca è case-sensitive. Nel frontend i risultati appaiono nella sidebar divisi per tipo (Progetto / Conversazione / Messaggio); cliccando su un risultato si apre direttamente la risorsa corrispondente.

Preferiti e archiviazione

Ogni conversazione può essere classificata come preferita (`isFavorite`) o archiviata (`isArchived`). Lo stato viene riportato nelle colonne `is_favorite` e `is_archived` e aggiornato

tramite PATCH /api/conversations/:id. Tre pulsanti filtro (Tutte / Preferite / Archivate) permettono di filtrare la lista; le conversazioni archiviate sono nascoste di default.

Eliminazione con conferma

L'eliminazione di progetti e conversazioni richiede un doppio click: la prima pressione cambia testo e stile del pulsante, la seconda esegue la DELETE.

7. Scelte implementative e limitazioni

- Ricerca: le tre tabelle vengono caricate in memoria con SELECT separate e collegate in JavaScript con find ().
- cercaStringa: confronto carattere per carattere usando .length e l'accesso per indice.
- codificaQuery: la codifica dei caratteri speciali è implementata manualmente con un ciclo if/else.
- Risposta: rispostaBot sceglie sempre la prima regola corrispondente.
- Ricerca case-sensitive: conseguenza diretta di cercaStringa, che confronta i caratteri.
- Lista conversazioni nell'area principale: i requisiti indicano che la sidebar deve mostrare i progetti e dare accesso alle conversazioni. La lista delle conversazioni e i filtri (Tutte / Preferite / Archivate) sono stati spostati nell'area principale anzichè nella sidebar, per sfruttare meglio lo spazio disponibile e rendere la navigazione più leggibile.
- Nessuna autenticazione: tutti i progetti sono visibili a chiunque apra l'applicazione.

8. Conclusioni

Avevo già avuto modo di avvicinarmi alla programmazione web grazie alla mia partecipazione al gruppo IT di JEUD, ma non avevo mai realizzato un progetto strutturato come questo, con un backend REST, un database e un frontend che comunicano tra loro. È stata un'esperienza molto più completa rispetto a quanto avevo fatto in precedenza.

Ho voluto renderlo il più possibile simile a un'applicazione reale e, dunque, oltre ai requisiti obbligatori ho scelto di implementare la ricerca nello storico e la gestione di preferiti e conversazioni archiviate. Queste funzionalità non erano richieste per i progetti individuali, ma ho voluto approfondire la gestione dello stato lato frontend e il coordinamento tra più endpoint.